

Build e BuildConfig in OpenShift

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Distinguere i tre tipi di build in OpenShift: Source-to-Image (S2I), Docker e Custom
 - Creare e configurare una **BuildConfig** con sorgente Git e strategia S2I
 - Configurare trigger di build: **ImageChange**, **GitHubWebhook**, **GenericWebhook**, **ConfigChange**
 - Avviare, annullare e monitorare build con **oc start-build**, **oc cancel-build**, **oc logs**
 - Descrivere il ciclo di vita di un Build: **New** → **Pending** → **Running** → **Complete** | **Failed**
 - Configurare l'output di una build verso un **ImageStreamTag**
 - Integrare build S2I con repository Git privati su ARO tramite Secret SSH
-

Teoria e Concetti Chiave

Cos'è un BuildConfig

Un **BuildConfig** è un oggetto OpenShift che definisce il processo di build di un'immagine container. È il "template" della build: ogni esecuzione genera un oggetto **Build** con un numero progressivo (**myapp-build-1**, **myapp-build-2**, ecc.).

Componenti principali di un BuildConfig:

Campo	Descrizione
spec.source	Sorgente del codice (Git, Binary, Dockerfile inline, ConfigMap/Secret)
spec.strategy	Come costruire l'immagine (Source/S2I, Docker, Custom)
spec.output	Dove pubblicare l'immagine risultante (ImageStreamTag, DockerImage)
spec.triggers	Cosa avvia automaticamente una build
spec.resources	Risorse CPU/memoria per il Pod di build

Tipi di Build: Strategia

Source-to-Image (S2I)

La strategia **S2I** è la più comune in OpenShift. Usa una **builder image** che sa come compilare e assemblare il codice sorgente. Il processo è:

1. Il Pod di build clona il repository Git
2. Inietta il codice sorgente nella builder image (es. **ubi9/openjdk-21**)
3. Esegue lo script **assemble** della builder image (compila, scarica dipendenze)
4. Produce una nuova immagine con lo script **run** pre-configurato
5. Fa push dell'immagine nel registry (ImageStream)

Nessun Dockerfile richiesto — la builder image gestisce tutto.

Builder image ufficiali Red Hat (S2I):

Builder Image	Linguaggio	Tag
registry.access.redhat.com/ubi9/openjdk-21	Java 21	1.19
registry.access.redhat.com/ubi9/nodejs-20	Node.js 20	1
registry.access.redhat.com/ubi9/python-311	Python 3.11	1
registry.access.redhat.com/ubi9/php-82	PHP 8.2	1

Docker Build

La strategia **Docker** usa un **Dockerfile** presente nel repository (o inline nel BuildConfig). Richiede che il Dockerfile sia presente nel repository e che la build venga eseguita con le stesse regole dei Dockerfile standard.

⚠️ Attenzione: La strategia Docker esegue comandi **RUN** nel Pod di build con privilegi limitati. Alcuni Dockerfile che funzionano con Docker Desktop non funzionano con OpenShift Build senza modifiche (es. **RUN apt-get** richiede **root**, configurabile tramite SCC).

Custom Build

La strategia **Custom** usa una builder image arbitraria definita dall'utente. È la più flessibile ma richiede la creazione di una builder image personalizzata. È rara in produzione.

Trigger di Build

Tipo di Trigger	Quando si attiva	Configurazione
ImageChangeTrigger	Quando l'immagine sorgente S2I (builder image) viene aggiornata	type: ImageChange con imageChange.from
GitHubWebhookTrigger	Al push su GitHub/GitLab tramite webhook HTTPS	type: GitHub con github.secret
GenericWebhookTrigger	Chiamata HTTP POST a un URL OpenShift	type: Generic con generic.secret
ConfigChangeTrigger	Quando la BuildConfig stessa viene modificata	type: ConfigChange

💡 Suggerimento: Per ambienti di produzione, combinare il **GitHubWebhookTrigger** (avvia la build al push) con il **ImageChangeTrigger** (avvia la build quando la base image viene aggiornata con patch di sicurezza).

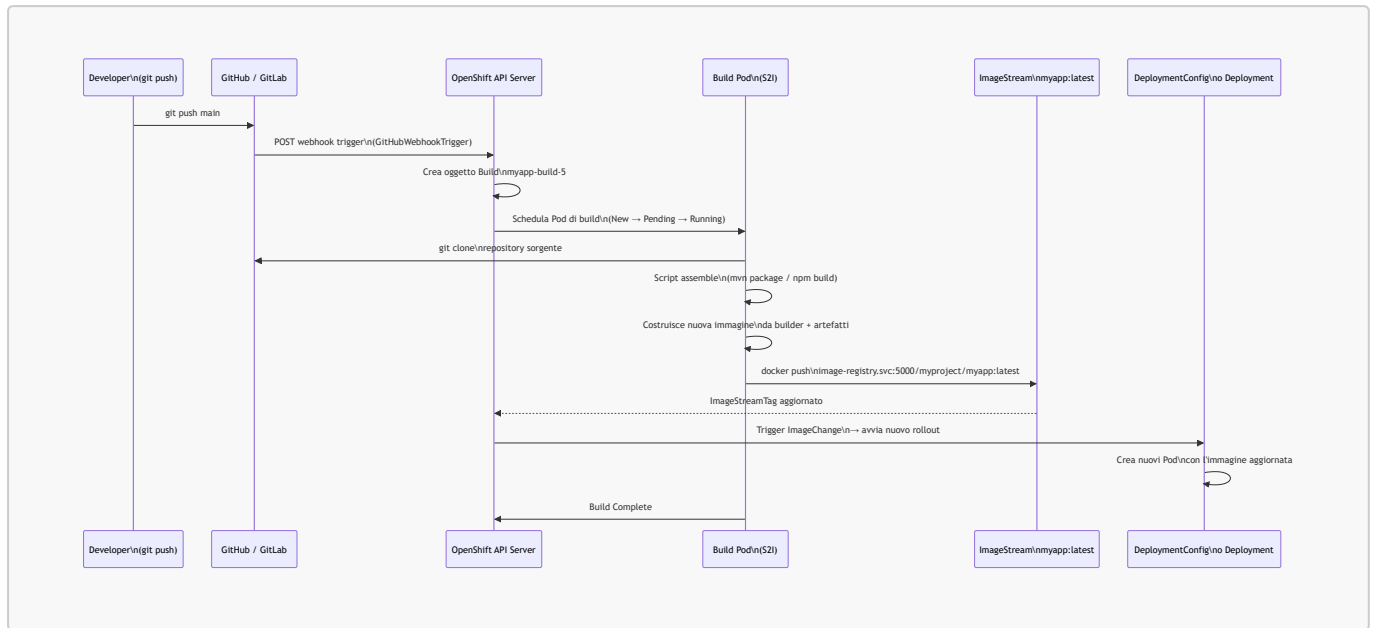
Ciclo di Vita di un Build

```
New → Pending → Running → Complete
                        → Failed
                        → Cancelled (da oc cancel-build)
                        → Error (errore infrastrutturale)
```

Stato	Descrizione
New	Build appena creata, non ancora schedulata
Pending	In attesa di un Pod di build disponibile
Running	Pod di build in esecuzione
Complete	Build completata con successo, immagine pubblicata
Failed	Build fallita (errore di compilazione, test, ecc.)
Cancelled	Build annullata manualmente
Error	Errore infrastrutturale (es. impossibile schedulare il Pod)

Diagrammi

Ciclo di Vita di una Build S2I



Comandi Pratici con Output Atteso

```
# Creare un'applicazione con BuildConfig S2I da repository Git pubblico
oc new-app \
  registry.access.redhat.com/ubi9/nodejs-20~https://github.com/sclorg/nodejs-
ex.git \
  --name=myapp \
  -n myproject

# Output atteso:
# --> Found image ubi9/nodejs-20 (nodejs-20 builder)
# --> Creating resources ...
#   imagestream.image.openshift.io "nodejs-20" created
#   imagestream.image.openshift.io "myapp" created
#   buildconfig.build.openshift.io "myapp" created
#   deployment.apps "myapp" created
#   service "myapp" created
# --> Success
#   Build scheduled, use 'oc logs -f buildconfig/myapp' to track its progress.
```

```
# Monitorare i log della build in corso
oc logs -f buildconfig/myapp -n myproject

# Output atteso:
# Cloning "https://github.com/sclorg/nodejs-ex.git" ...
# Commit: a1b2c3d (feat: update dependencies)
# ---> Installing application source ...
# ---> Building your Node application from source
# npm install ...
# added 127 packages in 12s
# ---> Build completed successfully
# Pushing image image-registry.openshift-image-
registry.svc:5000/myproject/myapp:latest ...
# Push successful
```

```
# Verificare lo stato dei Build creati
oc get builds -n myproject

# Output atteso:
```

NAME	TYPE	FROM	STATUS	STARTED	DURATION
myapp-1	Source	Git@a1b2c3d	Complete	2 minutes ago	1m30s

```
# Avviare manualmente una nuova build
oc start-build myapp -n myproject
```

```
# Output atteso:
# build.build.openshift.io/myapp-2 started
```

```
# Avviare una build con un commit specifico del repository
oc start-build myapp \
  --commit=main \
  --follow \
  -n myproject

# Output atteso:
# build.build.openshift.io/myapp-3 started
# Cloning "https://github.com/sclorg/nodejs-ex.git" ...
# ...
# Push successful
```

```
# Annullare una build in corso
oc cancel-build myapp-3 -n myproject

# Output atteso:
# build.build.openshift.io/myapp-3 cancelled
```

```
# Descrivere una BuildConfig per vedere tutti i trigger e la configurazione
oc describe buildconfig myapp -n myproject

# Output atteso:
# Name:          myapp
# Namespace:     myproject
# ...
# Strategy:      Source
# URL:           https://github.com/sclorg/nodejs-ex.git
# From Image:    ImageStreamTag ubi9/nodejs-20:latest
# Output to:     ImageStreamTag myapp:latest
# ...
# Triggered by:  Config, ImageChange
```

```
# Ottenere l'URL del webhook GitHub per la BuildConfig
oc describe buildconfig myapp -n myproject | grep -A 2 "Webhook GitHub"

# Output atteso:
# Webhook GitHub:
#   URL:
https://api.myaro.eastus.aroapp.io:6443/apis/build.openshift.io/v1/namespaces/myproject/buildconfigs/myapp/webhooks/<secret>/github
```

```
# Avviare una build da sorgente binario locale (cartella locale)
oc start-build myapp \
  --from-dir=./target \
  --follow \
  -n myproject

# Output atteso:
# Uploading directory "target" as binary input for the build ...
# build.build.openshift.io/myapp-4 started
# ...
# Push successful
```

Esempi YAML Completi

BuildConfig S2I con Source Git, Trigger, Output ImageStream, Risorse

```
# BuildConfig: definisce come costruire l'immagine myapp
# Ogni esecuzione crea un oggetto Build con numero progressivo
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: myapp-build
  namespace: myproject
  labels:
    app: myapp
spec:
  # Sorgente del codice
  source:
    type: Git # ← tipo di sorgente: Git | Binary |
              Dockerfile | None
    git:
      uri: https://github.com/myorg/myapp.git # ← URL del repository Git
      ref: main # ← branch, tag o commit da clonare
      contextDir: / # ← directory nel repo che contiene il
                    codice
      # Secret per accedere a repository privati (SSH o HTTP Basic)
      # sourceSecret:
      #   name: git-ssh-secret # ← nome del Secret con la chiave SSH

  # Strategia di build
  strategy:
    type: Source # ← Source (S2I) | Docker | Custom
    sourceStrategy:
      # Immagine builder S2I
      from:
        kind: ImageStreamTag # ← punta all'IS della builder image
        name: nodejs-20:latest # ← NodeJS 20 builder image
        namespace: openshift # ← namespace dove si trova l'IS
    builder (predefinito)

    # Variabili d'ambiente disponibili durante la build (script assemble)
    env:
      - name: NPM_MIRROR # ← mirror npm per build in ambienti
        offline
        value: "https://registry.npmjs.org"
      - name: NODE_ENV
        value: production

    # Aggiornare l'immagine builder prima della build
    forcePull: false # ← false = usa la cache locale della
    builder image

    # Output: dove viene pubblicata l'immagine risultante
```

```

output:
  to:
    kind: ImageStreamTag          # ← pubblica nell'ImageStream locale
    name: myapp:latest            # ← formato <imagestream>:<tag>

# Trigger di build automatici
triggers:
# Avvia una nuova build quando la BuildConfig viene modificata
- type: ConfigChange

# Avvia una nuova build quando la builder image (nodejs-20:latest) viene
aggiornata
- type: ImageChange
  imageChange:
    from:
      kind: ImageStreamTag
      name: nodejs-20:latest
      namespace: openshift

# Avvia una nuova build tramite webhook GitHub
- type: GitHub
  github:
    secret: my-github-webhook-secret # ← token segreto incluso nell'URL del
webhook                                     # ← NON usare lo stesso valore in più

BuildConfig

# Avvia una nuova build tramite webhook generico (es. Jenkins, GitLab, Gitea)
- type: Generic
  generic:
    secret: my-generic-webhook-secret
    allowEnv: false                # ← false = non permette di passare env
var via webhook

# Limite di build simultanee e storico dei Build da conservare
successfulBuildsHistoryLimit: 5    # ← conserva gli ultimi 5 Build
completati
failedBuildsHistoryLimit: 3        # ← conserva gli ultimi 3 Build falliti
runPolicy: Serial                  # ← Serial | Parallel |
SerialLatestOnly                  # ← SerialLatestOnly: se ci sono più
build in coda,                    # esegue solo l'ultima e cancella le
precedenti

# Risorse per il Pod di build (importante per build pesanti come Java/Maven)
resources:
  requests:
    cpu: 200m                      # ← minimo garantito
    memory: 512Mi
  limits:
    cpu: 1000m                    # ← massimo consentito
    memory: 2Gi

```

```
# Timeout della build in secondi (default: 600 = 10 minuti)
completionDeadlineSeconds: 900      # ← 15 minuti: aumentare per build
Maven lente
```

Note Specifiche per Azure ARO

Build con Repository Git Privati su ARO

Per clonare da repository Git privati (es. Azure DevOps, GitHub privato), è necessario un Secret con le credenziali SSH o HTTP Basic:

```
# Creare un Secret con una chiave SSH per repository privati
oc create secret generic git-ssh-secret \
  --from-file=ssh-privatekey=${HOME}/.ssh/id_ed25519 \
  --type=kubernetes.io/ssh-auth \
  -n myproject

# Annotare il Secret per associarlo all'host Git (opzionale ma consigliato)
oc annotate secret git-ssh-secret \
  "build.openshift.io/source-secret-match-uri-1=ssh://git@github.com/*" \
  -n myproject

# Applicare il Secret alla BuildConfig (via patch)
oc patch buildconfig myapp-build \
  --type=merge \
  --patch='{ "spec": { "source": { "sourceSecret": { "name": "git-ssh-secret" } } } }' \
  -n myproject
```

Build con Output verso Azure Container Registry (ACR)

Su ARO, è possibile configurare una BuildConfig per pubblicare l'immagine direttamente in ACR invece dell'ImageStream locale:

```
# Creare il Secret con le credenziali ACR per il push
oc create secret docker-registry acr-push-secret \
  --docker-server=myaroregistry.azurecr.io \
  --docker-username=$(az acr credential show --name myaroregistry --query username \
  -o tsv) \
  --docker-password=$(az acr credential show --name myaroregistry --query \
  "passwords[0].value" -o tsv) \
  -n myproject

# Associare il Secret al ServiceAccount "builder" (usato dai Pod di build)
oc secrets link builder acr-push-secret --for=mount -n myproject
```

```
# Output della BuildConfig verso ACR (invece di ImageStreamTag)
output:
  to:
    kind: DockerImage                # ← push diretto a registry esterno
    name: myaroregistry.azurecr.io/myapp:latest
  pushSecret:
    name: acr-push-secret           # ← Secret con credenziali push ACR
```

ARO Build Pods sui Worker Nodes

Su ARO, i Pod di build vengono eseguiti sui **worker node standard** (non su nodi dedicati). Se le build Java/Maven richiedono molte risorse, considerare:

```
# Verificare su quale nodo è stato schedulato il Pod di build
oc get build myapp-1 -n myproject \
  -o jsonpath='{.status.podName}'

# Quindi:
oc get pod myapp-1-build -n myproject \
  -o jsonpath='{.spec.nodeName}'

# Output atteso:
# aro-worker-eastus-1
```

Riepilogo dei Punti Chiave

- La strategia **S2I** (Source-to-Image) è la più comune su OpenShift: usa una builder image Red Hat per compilare il codice sorgente senza richiedere un Dockerfile
 - La strategia **Docker** usa un Dockerfile presente nel repository; la strategia **Custom** usa una builder image arbitraria
 - Un **BuildConfig** è il template; ogni esecuzione crea un oggetto **Build** con numero progressivo
 - I trigger automatici includono **ImageChange** (nuova builder image), **GitHub/Generic** (webhook), **ConfigChange** (modifica del BC)
 - **runPolicy: SerialLatestOnly** è utile per evitare build obsolete in coda: solo l'ultima build viene eseguita
 - Il campo **spec.output.to** specifica dove viene pubblicata l'immagine: **ImageStreamTag** (registry interno) o **DockerImage** (registry esterno come ACR)
 - Per repository privati su ARO, usare un Secret **kubernetes.io/ssh-auth** associato alla BuildConfig
 - I Pod di build su ARO girano sui worker node standard; impostare **resources.limits** adeguati per build intensive
-

Domande di Verifica

1. Qual è la differenza tra la strategia **Source** (S2I) e la strategia **Docker** in una BuildConfig?

- A) **Source** usa un Dockerfile, **Docker** usa una builder image S2I
- B) **Source** usa una builder image S2I senza richiedere un Dockerfile; **Docker** usa un Dockerfile presente nel repository ☒
- C) **Source** compila solo codice Java, **Docker** compila qualsiasi linguaggio
- D) Non c'è differenza: entrambe usano un Dockerfile

2. Quale campo della BuildConfig controlla se le build vengono eseguite in sequenza o in parallelo?

- A) `spec.strategy.type`
- B) `spec.completionDeadlineSeconds`
- C) `spec.runPolicy` ☒
- D) `spec.triggers[].type`

3. Qual è il ciclo di vita corretto di un Build OpenShift?

- A) Pending → New → Running → Complete
- B) New → Pending → Running → Complete | Failed | Cancelled ☒
- C) Created → Scheduled → Building → Done
- D) Init → Active → Succeeded | Error

4. Quale tipo di trigger avvia automaticamente una nuova build quando la builder image S2I viene aggiornata nel suo ImageStream?

- A) `ConfigChange`
- B) `Generic`
- C) `GitHub`
- D) `ImageChange` ☒

5. Per inviare il codice sorgente locale (cartella `./target`) a una build OpenShift senza usare un repository Git, quale comando si usa?

- A) `oc new-build --from-dir=./target`
- B) `oc start-build myapp --from-dir=./target` ☒
- C) `oc apply -f ./target`
- D) `oc upload myapp ./target`